

智能系统设计实践

六足机器人避障控制 第一次进展汇报

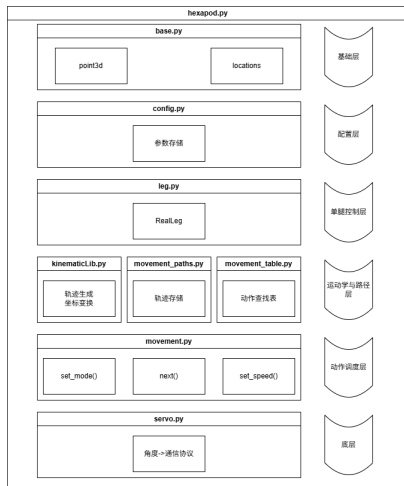
谭诗弘，黄育民，曹洪琿

武汉大学电子信息学院

2025 年 9 月 29 日



控制流程图



基础层

base.py

定义了 3D 点 (point3d) 和腿尖位置集合 (locations), 作为六足机器人所有运动的几何基础。

- point3d: 描述一个点 (x,y,z) , 支持加减乘法。
- locations: 保存 6 条腿的足尖坐标, 方便批量操作。

提供了坐标系的数据结构, 便于后续的运动轨迹和逆运动学操作。

配置层

config.py

存放机器人硬件和运动学的固定参数。

- 关节长度、安装角度、腿的初始坐标。
- 默认待机位置。
- 预定义的步态轨迹路径点。

通过这些参数计算每条腿运动的范围和姿态。

单腿控制层

leg.py → RealLeg

- 将每条腿视为一个对象，将全局坐标系下的目标点转化为腿的局部坐标系。
- 调用 kinematicLib.py 计算舵机角度
- 调用 servo.py 将角度转化为 PWM 命令

负责具体的单腿控制部分。

运动学与路径层

kinematicLib.py

负责逆运动学操作，将三维点转换为舵机角度。同时生成半圆/椭圆腿尖路径，并处理旋转矩阵的坐标变换。

movement_paths.py

存储不同运动模式的轨迹（如前进、后退、旋转等），每个轨迹有 20 个时间步构成，每一步包含六条腿的三维坐标。

movement_table.py

存储一个动作查找表，将各种运动模式映射到对应的路径点序列，避免每次都实时计算轨迹，加快执行速度。

动作调度层

movement.py

作为动作控制器，负责管理运动模式和路径

- set_mode(): 切换机器人的工作模式。
- set_speed(): 调整机器人的运动速度。
- next(): 取出下一步的运动坐标，供六条腿执行。

其决定机器人下一时刻的运动状态。

底层

servo.py

负责将运动数据转化为舵机通信协议数据帧，通过串口等发送到实际硬件，驱动硬件执行。
负责真实动作的运行。

六足整体协调

hexapod.py→Hexapod 类

总体协调各个模块的工作。

- 初始化六条腿 (RealLeg)。
- 接受用户的运动指令 (move())。
- 调用 movement.py 生成轨迹点。
- 将轨迹点分配给各条腿，调用 leg.move_tip()。
- 保持整体的平衡和同步。

控制流程总结

- ① 用户输入（例如 `hexapod.move(10,0,0)` → 前进 10cm）。
- ② `movement.py` 根据输入调用 `movement_table` / `pathPlanning` 生成完整轨迹点。
- ③ `hexapod.py` 轮询调用 `movement.next()` → 取出 6 条腿的坐标。
- ④ `leg.py` 将目标点 → 逆运动学角度。
- ⑤ `servo.py` 把角度 → 舵机指令。
- ⑥ 舵机动作，机器人移动。

始终保持三条腿支撑、三条腿摆动的三角步态，从而保证平衡。

Thanks!